

Guía de ejercitación

Funcionamiento de los lenguajes orientados a objetos

Prof. Pablo Eduardo Hernández · 2026

BLOQUE 1 La Java Virtual Machine — JDK, JRE, JVM, bytecode y Classpath

Ejercicio 1.1 · Opción múltiple

Marcá la única opción correcta en cada caso.

- 1) ¿Qué se necesita instalar para poder compilar código Java?
 - a) El JRE, porque incluye el compilador.
 - b) El JDK, porque incluye a javac y demás herramientas.
 - c) La JVM, porque traduce el código fuente.
 - d) Solo el Classpath.
- 2) El archivo que genera el compilador de Java tiene extensión:
 - a) .java
 - b) .class
 - c) .exe
 - d) .byte
- 3) ¿Qué componente ejecuta el bytecode en cada plataforma?
 - a) El compilador javac.
 - b) La JVM (Java Virtual Machine).
 - c) Directamente el sistema operativo.
 - d) Directamente el hardware.
- 4) La frase “write once, run anywhere” es posible porque:
 - a) Java se compila a código de máquina específico de cada SO.
 - b) El mismo bytecode lo ejecuta la JVM de cada plataforma.
 - c) Java no se compila, se interpreta línea por línea desde el .java.
 - d) El JRE reescribe el código fuente para cada sistema.
- 5) El CLASSPATH es:
 - a) Una carpeta obligatoria llamada así dentro del proyecto.
 - b) Una variable de entorno del SO que indica dónde buscar clases y librerías.
 - c) Un método de la clase Object.
 - d) Otro nombre del compilador.

Ejercicio 1.2 · Verdadero o Falso

Escribí V o F. Si es falsa, corregila.

- __V__ La JVM es parte del JRE.
- __V__ El JDK incluye al JRE.
- __F__ En Java se compila un binario distinto para cada sistema operativo.
- __v__ El compilador de Java se llama javac.
- __f__ El bytecode se ejecuta directamente sobre el hardware, sin intermediarios.
- __v__ El Garbage Collector se encarga de liberar la memoria que ya no se usa.

Ejercicio 1.3 • Relacionar columnas

Uní cada término con su descripción.

JDK	Kit de desarrollo: compilador, generador de documentación y herramientas.
JRE	Entorno de ejecución: lo mínimo para correr un programa Java ya compilado.
JVM	Máquina virtual que ejecuta el bytecode sobre cada plataforma.
javac	Programa que compila el código fuente a bytecode.
.class	Archivo que contiene el bytecode generado.
Classpath	Variable de entorno donde se buscan las clases y librerías.

Ejercicio 1.4 • Ordenar el proceso

Ordená del 1 al 6 las etapas desde el código fuente hasta el hardware:

- __3__ Bytecode (.class)
- __2__ Compilador (javac)
- __6__ Hardware
- __1__ Archivo fuente (.java)
- __5__ Sistema Operativo
- __4__ JVM (ClassLoader, JIT, ejecución)

Ejercicio 1.5 • Respuesta breve

- 1) Explicá con tus palabras por qué se dice que Java es “write once, run anywhere” y en qué se diferencia de un lenguaje como C.

Se lo llama de esa manera debido a que Java se puede compilar y ejecutar en cualquier hardware que tenga su JVM, debido a que la misma aporta una abstracción al hardware y al software de plataforma. Otros lenguajes como C, no poseen esta característica, lo que deriva en que probablemente requiera de, por ejemplo, emuladores de software de plataforma para poder ser ejecutado en un hardware diferente al nativo.

- 2) ¿Qué problema podés tener si el CLASSPATH no está bien configurado?

Si está mal configurado, Java no va a saber en qué carpetas buscar los archivos y librerías que tu código necesita. Como resultado, cuando quieras compilar o ejecutar el programa, te va a tirar error diciendo que no encuentra las clases necesarias para funcionar.

BLOQUE 2

Estructura de un programa y convenciones de nomenclatura

Ejercicio 2.1 · Nombres autoexplicativos

Reescribí cada nombre para que sea autoexplicativo, pronunciable y con una distinción significativa. Indicá qué representaría.

```
Int edadUsuario;  
  
double  
precioUnitario;  
  
boolean activo;  
String email;  
  
void  
generarRoporte {  
... }
```

Ejercicio 2.2 · camelCase vs PascalCase

Corregí cada identificador según la convención que le corresponde (clases en PascalCase; variables y métodos en lowerCamelCase).

```
clase: CuentaBancaria  
variable: saldoTotal  
  
método: salcularInteres  
  
clase: Factura  
  
variable: nombreCliente
```

Ejercicio 2.3 · Palabras reservadas

Indicá qué líneas NO compilan y explicá por qué.

```
int public = 10; NO (public  
es palabra reservada)  
  
int contador = 0; SI  
  
double for = 3.14; NO (For  
es palabra reservada)  
  
String nombre = "Ana"; SI  
  
boolean int = true; NO  
(Int es palabra reservada)
```

Ejercicio 2.4 · Estructura básica

Completá los huecos para obtener un programa que imprima "Hola mundo".

```
__public__ class Hola {  
    public __static__ void main(String[] args) {  
        System.out.__println__("Hola mundo");  
    }  
}
```

Ejercicio 2.5 • Comentarios

Agregá un comentario de bloque que describa la clase y un comentario de línea sobre el cálculo.

```
public class Circulo { //Cálculo de área de un círculo a
    partir del radio.
    double radio = 5;
    double area = 3.14 * radio * radio; //Ecuacion de
        área
}
```

BLOQUE 4

Arreglos — el arreglo como objeto, posiciones, acceso

y recorrido

Ejercicio 3.1 • Declaración

Escribí la declaración y creación de cada arreglo:

- 1) Un arreglo de 8 números enteros. `Int[] números = new int[8]`
- 2) Un arreglo de 5 objetos de tipo `Producto`. `Producto[] productos = new Producto[5]`
- 3) Un arreglo de 3 valores booleanos. `Boolean[] estados = new boolean[3]`

Ejercicio 3.2 • ¿Qué imprime?

Indicá la salida exacta de cada fragmento.

a) Fragmento 1:

```
int[] valores = new int[4];
valores[1] = 10;
valores[3] = 25;
for (int i = 0; i < valores.length; i++) {
    System.out.println(valores[i]);
}
```

Imprime:

0
10
0
25

b) Fragmento 2:

```
String[] nombres = new String[3];
nombres[0] = "Ana";
for (int i = 0; i < nombres.length; i++) {
    System.out.println(nombres[i]);
}
```

Imprime:

Ana
null
null

BLOQUE 5

Ejercicio 3.3 · Detectá el error

Cada línea tiene un problema. Indicá cuál y por qué.

```
int[] a = new int[5]; a[5] = 3;    // la pos 5 no pertenece al arreglo
System.out.println(a.length()); //length no es un metodo, sobran
parentesis
Auto[] autos = new Auto[2]; autos[0].acelerar(); //Intenta utilizar un
método en donde no existe un objeto auto (pos 0).
```

Ejercicio 3.4 · Escribí el código

Dado int[] numeros ya cargado, escribí el fragmento que resuelve cada punto:

- 1) Sumar todos los elementos e imprimir el total.

```
int suma = 0;
for (int num : numeros) {
    suma += num;
}
System.out.println(suma);
```

- 2) Encontrar e imprimir el valor máximo.

```
int max = numeros[0]; // Asumimos que el arreglo no está vacío
for (int i = 1; i < numeros.length; i++) {
    if (numeros[i] > max) {
        max = numeros[i];
    }
}
System.out.println(max);
```

- 3) Contar cuántos elementos son mayores a 10.

```
int contador = 0;
for (int num : numeros) {
    if (num > 10) {
        contador++;
    }
}
System.out.println(contador);
```

BLOQUE 6

Ejercicio 3.5 · Primitivos vs. referencia

- 1) Indicá el valor por defecto de cada posición recién creado el arreglo: `int[]`: 0, `double[]`: 0.0, `boolean[]`: false, `String[]`: null.
- 2) En un arreglo de objetos, ¿qué guarda realmente cada posición: el objeto o una referencia? Explicá.

Guarda una referencia (dirección en memoria) que apunta a la ubicación en memoria del objeto en cuestión.

Registro de temperaturas

Actividad grupal que integra los tres bloques. Se entrega el código fuente comentado y una breve explicación de las decisiones de diseño.

Consigna

Escribí una clase `RegistroTemperaturas` que trabaje con las temperaturas de una semana (7 días) usando un arreglo de `double`. La clase debe permitir:

1. Cargar las 7 temperaturas del arreglo.
2. Calcular e imprimir el promedio de la semana.
3. Encontrar la temperatura máxima y la mínima.
4. Contar cuántos días superaron un umbral dado (por ejemplo, 30.0 grados).
5. Recorrer el arreglo usando `length` e imprimir cada día con su temperatura.

Requisitos de calidad

- Nombres autoexplicativos y pronunciables; clase en PascalCase, variables y métodos en camelCase.
- Ningún identificador que sea palabra reservada.
- Al menos un comentario de bloque (descripción de la clase) y comentarios de línea donde aporten.
- Todo el recorrido de arreglos debe usar `length` (no números fijos).